INFOTACT SOLUTIONS | JAVA DEVELOPMENT INTERNSHIP

AeroSaga

Temporal.io Autonomous Drone Mission Control

Build, Test and Go-Live Guide
Every step from an empty laptop to a live HTTPS product, with a checkpoint after each step and a fix list for every common failure.

Item	Value
Project	AeroSaga: Temporal.io Autonomous Drone Mission Control
Ref. No.	ITS/JD/1144
Intern	Ganesh Chakravarthula, Associate L1 Intern, Java Development
Project window	10 Oct 2026 to 10 Nov 2026 (1 month)
Guide date	14 Oct 2026
Source code	`aerosaga-v1.0.zip` (comes with this guide)
Stack	Java 21, Spring Boot 4.1.1, Temporal Java SDK 1.38.0, PostgreSQL 17, Docker, Caddy

How to use this guide
Do the steps in order. Each step ends with a green CHECKPOINT box.
Do not start the next step until the checkpoint passes. Skipping ahead hides the real cause of later errors.
If a checkpoint fails, use the red **If it fails** table under that step. Fix the first error first.
Commands in grey boxes are typed exactly as shown. Windows and Mac versions are labelled.

Contents

0. Read this first . . . 4

1. What you are building . . . 5

2. Your 4-week plan . . . 8

Part A. Install the software . . . 9

A1 Open a terminal . . . 9

A2 Install Java 21 (JDK) . . . 9

A3 Install Git and set your name . . . 10

A4 Install IntelliJ IDEA . . . 11

A5 (Optional) Install Maven for the command line . . . 11

A6 Install the Temporal CLI . . . 12

A7 Install Docker Desktop . . . 12

A8 Create a GitHub account and an access token . . . 13

A9 Final install check . . . 13

Part B. Get the project and prove it builds . . . 14

B1 Unzip the project to a safe folder . . . 14

B2 Start version control and push to a private GitHub repository . . . 14

B3 Open the project in IntelliJ . . . 15

B4 Run all 59 tests (the real proof) . . . 16

B5 Commit the first green build . . . 17

Part C. Run it on your laptop . . . 18

C1 Start the Temporal server . . . 18

C2 Start the application . . . 18

C3 Log in to the dashboard . . . 19

C4 Fly a normal mission . . . 19

C5 Test every fault scenario . . . 20

C6 Crash tests: prove durability . . . 21

C7 Call the REST API directly . . . 21

C8 Code tour: understand what you will present . . . 22

C9 Commit progress . . . 23

Part D. Run the production stack on your laptop (Docker) . . . 25

D1 Create the secrets file . . . 25

D2 Build and start all containers . . . 25

D3 Use the stack through HTTPS . . . 26

D4 Stop the stack . . . 26

Part E. Go live on a cloud server (Azure for Students) . . . 28

E1 Activate Azure for Students . . . 28

E2 Create the virtual machine . . . 28
E3 Give the server a DNS name . . . 29
E4 Connect to the server with SSH . . . 29
E5 Prepare the server (updates and Docker) . . . 30
E6 Download your code to the server . . . 30
E7 Create production secrets . . . 31
E8 Build and start the production stack . . . 32
E9 Verify the live site . . . 32
E10 Open the Temporal UI safely (SSH tunnel) . . . 33
E11 Automatic daily backups and a restore test . . . 33
E12 Uptime monitoring and cost control . . . 34
E13 Release gate (go-live checklist) . . . 34

Part F. Operate the live product (runbook) . . . 36

F1. Deploy an update . . . 36
F2. Roll back a bad update . . . 36
F3. Logs, health, resources . . . 36
F4. Change passwords . . . 36
F5. Incident checklist (site down) . . . 36
F6. Weekly routine (10 minutes) . . . 37

Part G. Internship deliverables . . . 38

G1 Documentation package . . . 38
G2 Write the project report . . . 38
G3 Build the presentation (12 slides) . . . 39
G4 Demo script (6 minutes) . . . 39
G5 Screenshot checklist . . . 40
G6 Final verification before submission . . . 40
G7 Likely review questions (prepare answers) . . . 41

Appendix A. Troubleshooting index . . . 42

Appendix B. Command cheat sheet . . . 43

Appendix C. Requirements traceability . . . 44

Appendix D. Configuration reference . . . 45

Appendix E. REST API reference . . . 46

0. Read this first

This guide turns the AeroSaga source code into a working product on your laptop, and then into a live website on a cloud server. It also covers the internship deliverables: documentation, report, presentation and demo.

0.1 Status words used in this guide

Word	Meaning
PASS	The checkpoint showed exactly the expected result.
FAIL	The checkpoint showed something else. Stop and use the fix table.
BLOCKED	You cannot continue because something outside your control is missing (account, network, approval).
NOT VERIFIED	Work exists but nobody has run the check yet. Never call this PASS.

0.2 What was already checked before you got this guide

Honest status, so you know where problems are most likely to appear:

Part	How it was checked	Result
Mission rules, routes, no-fly zones, drone fleet, battery model, flight simulator, state save/restore	42 unit tests compiled and run with Java 21	PASS (42 of 42)
Saga workflow logic: undo order, final outcomes, abort, retries, nothing left booked	18 checks run against a small stand-in for the Temporal library	PASS (18 of 18)
Dashboard JavaScript, YAML files, pom.xml, docker-compose.yml, backup script	Syntax checks (node, YAML parser, XML parser, docker compose config, bash -n)	PASS
Real Temporal library, Spring Boot wiring, database, security, 17 integration tests	Could not run: the build machine was not allowed to download Java libraries	NOT VERIFIED

What this means for you
Step **B4** (run all 59 tests) is the first time the full product runs. That step is the real proof.
If B4 shows a compile error or a failing test, do not change code randomly. Copy the FIRST error (the first red block), and follow the B4 fix table. If it is not listed, send that first error to your mentor or AI assistant together with the file name and line number.

0.3 Originality and understanding

- The internship terms say submitted work must be original, and you will present it. Read the code tour (section C8) and be able to explain every package in your own words.
- Make all commits yourself, under your own name (step B2). Your Git history is evidence of your work.
- Do not share company-confidential material in public. Keep the GitHub repository **private**.

1. What you are building

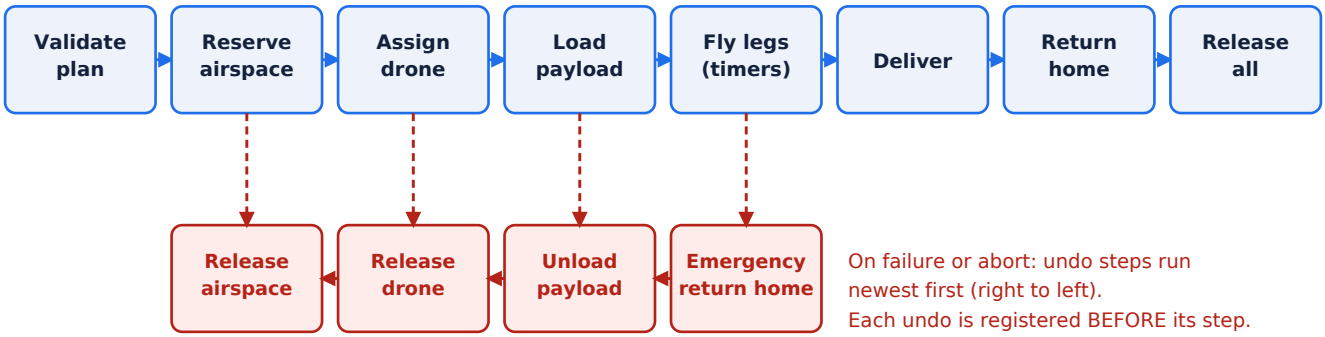
AeroSaga is mission control software for a fleet of delivery drones. The drones are simulated, so no hardware is needed, but the control logic is the same kind of logic real drone companies use.

1.1 The story of one mission

1. An operator opens the dashboard, clicks points on a map (waypoints), sets a payload weight, and presses Launch.
2. The system checks the plan: inside the allowed area, not over the airport, not too long, not too heavy.
3. It books an airspace slot (only 3 drones may fly at once).
4. It picks the drone with the most spare battery that can lift the payload.
5. It loads the payload, flies each leg, drops the payload at the last stop, and flies home.
6. It releases the drone and the airspace slot.

If something goes wrong at any point (battery fault, a no-fly zone appears, the operator presses Abort), the system **undoes every finished step in reverse order**: fly home, unload, release drone, release airspace. Nothing stays half-booked. This is called a **saga**.

Normal path (left to right). Each box is one activity; Temporal records each result.



1.2 Why Temporal

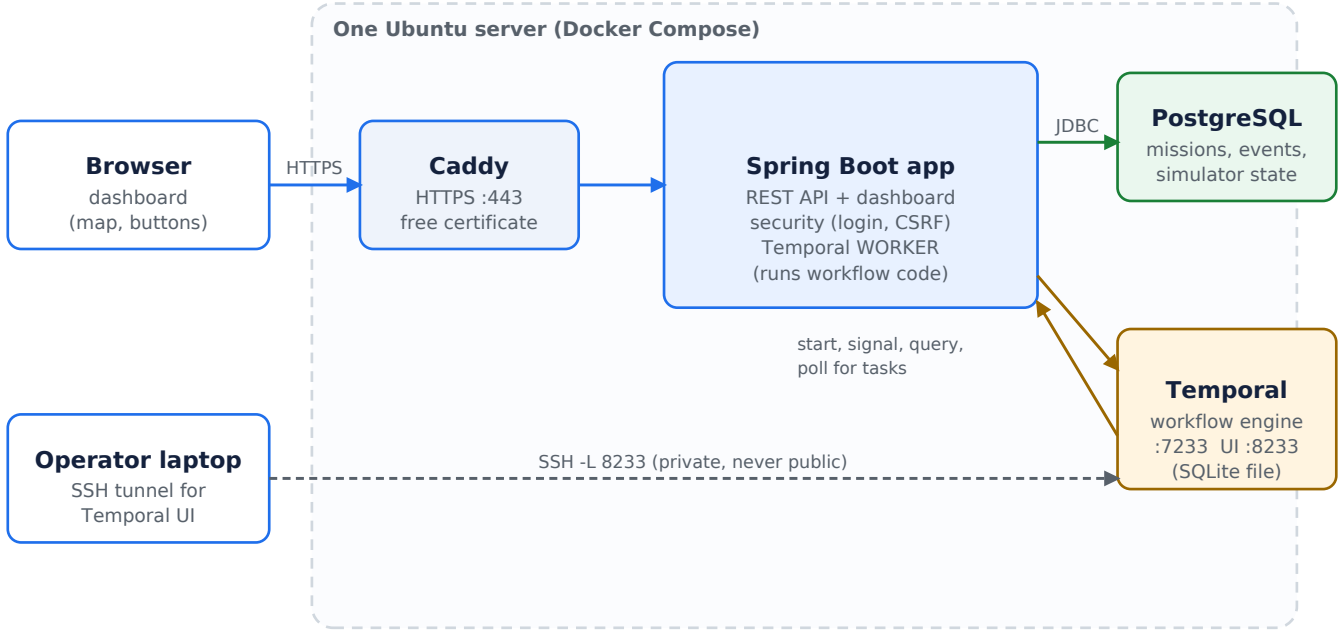
A mission can take many minutes. During that time the server may crash or restart. Normal code would lose track of the mission. **Temporal** is a workflow engine that writes down every finished step. If our app restarts, Temporal replays the mission code and continues from the exact step where it stopped. You will prove this yourself in step C6.

1.3 Words you need (glossary)

Word	Simple meaning
Workflow	The mission as step-by-step code (`MissionWorkflowImpl`). Must be deterministic: same input, same decisions.
Activity	One action that touches the outside world: book a drone, fly a leg, write to the database. Can fail and be retried.
Worker	A background loop inside our app that asks Temporal for work and runs workflow and activity code.
Task queue	A named line where Temporal puts work. Ours is `aerosaga-missions`.
Signal	A message sent INTO a running workflow. Our Abort button sends a signal.

Word	Simple meaning
Query	A read-only question to a running workflow. The dashboard asks for live status with a query.
Saga / compensation	Each step has an undo step. On failure, undo steps run newest first.
Idempotent	Safe to run twice. Booking a drone twice for the same mission still books only one drone.
Retry	Temporal runs a failed activity again with growing waits. Business failures (low battery) are not retried.
Flyway migration	A numbered SQL file that creates or changes database tables, run once, in order.
CSRF	An attack where another website makes your browser click our buttons. We block it with a secret token.
Reverse proxy (Caddy)	The front door on the server. Gets a free HTTPS certificate and forwards traffic to the app.

1.4 Architecture



- **Spring Boot app**: one Java program with the REST API, the dashboard pages, login, and the Temporal worker.
- **PostgreSQL**: stores mission history (for reports) and the simulator state (so drones survive restarts).
- **Temporal**: stores workflow progress. Single-node with a SQLite file. Enough for this project; for high availability a company would use Temporal Cloud or a full cluster.
- **Caddy**: HTTPS front door. Only ports 80 and 443 are public. Temporal UI stays private.

1.5 Fault scenarios (for testing and demo)

Scenario	What happens	Expected final state
NONE	Normal mission.	COMPLETED
LOW_BATTERY	Battery cell fault on leg 1 drains 70%. Drone cannot reach home with reserve.	ROLLED_BACK, code LOW_BATTERY, drone flown home

Scenario	What happens	Expected final state
GEOFENCE_BREACH	A temporary no-fly zone appears over waypoint 1. Drone refuses to enter.	ROLLED_BACK, code GEOFENCE_BREACH, drone never moved
FLAKY_TELEMETRY	Telemetry link fails twice on leg 1, then recovers.	COMPLETED (Temporal retried automatically)
Abort button	Operator aborts while in flight.	ABORTED, code OPERATOR_ABORT, everything undone

2. Your 4-week plan

Project window: 10 Oct to 10 Nov 2026. Today is 14 Oct. Follow this plan and keep 4 days of buffer.

Dates	Do these parts	Proof to show your mentor
14 to 17 Oct	Part A (install), Part B (tests pass), push to private GitHub	Screenshot: Tests run: 59, Failures: 0. GitHub repo link.
18 to 24 Oct	Part C (run locally, 5 scenarios, crash tests), code tour C8	Screenshots of dashboard and Temporal UI for each scenario.
25 to 31 Oct	Part D (Docker locally), Part E (go live on Azure), Part F (backups)	Live HTTPS link. Backup file listed.
1 to 6 Nov	Part G: documentation, report, presentation, demo rehearsal	Draft report and slides to mentor.
7 to 10 Nov	Buffer, final checklist (G6), submit	Final submission.

2.1 Weekly progress message (copy and fill)

```
Subject: AeroSaga weekly update - Week <n> (<dates>)

Done this week:
- <step IDs finished, e.g. A1-A9, B1-B6>
Evidence:
- <screenshot names / links>
Problems and how I fixed them:
- <problem> -> <fix>
Next week:
- <step IDs>
Blockers (need help):
- <none / describe>
```

Part A. Install the software

You install everything once. Each tool has its own checkpoint. Total time: about 60 to 90 minutes, mostly downloads. You need a laptop with at least 8 GB RAM and 15 GB free disk.

Tool	Why you need it	Needed for
Java 21 JDK (Temurin)	Compiles and runs the Java code	Everything
Git	Version control; the evaluation checks it	Everything
IntelliJ IDEA	Code editor with built-in Maven (build tool)	Parts B, C
Temporal CLI	Runs the workflow engine on your laptop	Part C
Docker Desktop	Runs the full production stack locally	Part D (optional but recommended)
GitHub account	Stores your code privately; the server downloads it from here	Parts B, E
SSH client	Connects to the cloud server (already built into Windows 10/11 and Mac)	Part E

A1 Open a terminal about 2 min

Why: Most steps need you to type commands.

Windows

1. Press the **Start** button.
2. Type **PowerShell**.
3. Click **Windows PowerShell**. A blue or black window opens. This is your terminal.

Mac

1. Press **Cmd + Space**.
2. Type **Terminal** and press Enter.

To run a command: type it (or paste it with right-click on Windows, Cmd+V on Mac) and press **Enter**. Wait until the prompt comes back before typing the next command.

Windows note: in PowerShell, always type `curl.exe` (not `curl`) in this guide. Plain `curl` is a different PowerShell command.

CHECKPOINT: PASS looks like this
A window with a blinking cursor, for example `PS C:\Users\ganesh>` (Windows) or `ganesh@MacBook ~ %` (Mac).

If it fails

What you see	Likely cause	Fix
winget not found later in A2	Old Windows App Installer	Open Microsoft Store, search **App Installer**, click Update. Reopen PowerShell.

A2 Install Java 21 (JDK) about 10 min

Why: Spring Boot 4 and Temporal need Java 17+; this project is set to Java 21.

Windows PowerShell

```
winget install EclipseAdoptium.Temurin.21.JDK
```

Mac (needs Homebrew from https://brew.sh first)

```
brew install --cask temurin@21
```

No winget or brew? Download the **JDK 21 .msi** (Windows) or **.pkg** (Mac) from https://adoptium.net and run it with default options. On Windows, in the installer, also enable **Set JAVA_HOME variable**.

Close the terminal, open a new one, then check:

```
java -version
```

CHECKPOINT: PASS looks like this
First line contains `openjdk version "21.` (for example 21.0.8). Any 21.x is fine.

If it fails

What you see	Likely cause	Fix
'java' is not recognized / command not found	Terminal opened before install finished, or PATH not updated	Close every terminal and open a new one. If still failing, reboot.
Shows version 1.8, 11 or 17	An older Java comes first in PATH	Uninstall old Java from Settings > Apps, or move the JDK 21 `bin` folder to the top of Path (A1 PATH help).
winget asks to accept terms	First use of winget	Type **Y** and Enter.

A3 Install Git and set your name about 5 min

Why: Every commit records who made it. Your evaluator will see this.

Windows PowerShell

```
winget install --id Git.Git -e
```

Mac

```
xcode-select --install
```

Open a NEW terminal and set your identity (use your real name and the email of your GitHub account):

```
git config --global user.name "Ganesh Chakravarthula"
git config --global user.email "you@example.com"
git --version
```

CHECKPOINT: PASS looks like this
`git --version` prints `git version 2.x`.

If it fails

What you see	Likely cause	Fix
'git' is not recognized	Terminal opened before install	Open a new terminal. Reboot if needed.
Wrong name in commits later	Typo in config	Run the two `git config` lines again with the correct values.

A4 Install IntelliJ IDEA about 15 min

Why: Editor plus built-in Maven, so you do not need to install Maven separately.

1. Open https://www.jetbrains.com/idea/download
2. Download **IntelliJ IDEA** for your system. The free version is enough. (Students can also get the paid version free at https://www.jetbrains.com/student.)
3. Run the installer with default options. On Windows tick **Add 'bin' folder to PATH** and **.java** file association.
4. Start IntelliJ once and finish the welcome screens.

CHECKPOINT: PASS looks like this
IntelliJ opens and shows the **Welcome to IntelliJ IDEA** window with **New Project** and **Open** buttons.

If it fails

What you see	Likely cause	Fix
Installer blocked by antivirus	Antivirus false alarm	Download again from the official site only, allow the installer.
Very slow on 8 GB RAM	Too many apps open	Close browsers and other apps while IntelliJ indexes the first time.

A5 (Optional) Install Maven for the command line about 10 min

Why: IntelliJ already contains Maven. Install this only if you want to type `mvn` in a terminal.

Mac

```
brew install maven
```

Windows

1. Open https://maven.apache.org/download.cgi and download the **Binary zip archive**.
2. Create the folder `C:\tools` and extract the zip there. You get a folder like `C:\tools\apache-maven-3.9.x`.
3. Add `C:\tools\apache-maven-3.9.x\bin` to Path. Windows: press Start, type **environment variables**, open **Edit the environment variables for your account**. Select **Path**, click **Edit**, click **New**, paste the folder, click **OK** on every window. Then close ALL terminal windows and open a new one.

```
mvn -version
```

CHECKPOINT: PASS looks like this
`Apache Maven 3.9.x` and `Java version: 21...` are printed.

If it fails

What you see	Likely cause	Fix
mvn shows Java 1.8 or 17	JAVA_HOME points to an old JDK	Set JAVA_HOME (user variable) to the JDK 21 folder, e.g. `C:\Program Files\Eclipse Adoptium\jdk-21...`. Reopen terminal.
'mvn' is not recognized	Path entry wrong	The Path entry must end in `\bin` and the folder must contain `mvn.cmd`.

A6 Install the Temporal CLI about 10 min

Why: It runs a local Temporal server and its web UI on your laptop.

Mac

```
brew install temporal
```

Windows

1. Open https://docs.temporal.io/cli/setup-cli and choose the **Windows** download (amd64 for most laptops). The direct link pattern is https://temporal.download/cli/archive/latest?platform=windows&arch=amd64
2. Extract the zip to `C:\tools\temporal` so that `C:\tools\temporal\temporal.exe` exists.
3. Add `C:\tools\temporal` to Path. Windows: press Start, type **environment variables**, open **Edit the environment variables for your account**. Select **Path**, click **Edit**, click **New**, paste the folder, click **OK** on every window. Then close ALL terminal windows and open a new one.

```
temporal --version
```

CHECKPOINT: PASS looks like this
A version line such as `temporal version 1.x.x` is printed.

If it fails

What you see	Likely cause	Fix
'temporal' is not recognized	Path entry missing or points to wrong folder	The Path folder must directly contain `temporal.exe`. Open a new terminal.
Windows SmartScreen blocks temporal.exe	Unsigned download warning	Click **More info** then **Run anyway** only if you downloaded from temporal.download.

A7 Install Docker Desktop about 20 min

Why: Needed for Part D (test the production stack on your laptop). The cloud server in Part E installs Docker separately.

Windows

1. Right-click Start, choose **Terminal (Admin)** or **Windows PowerShell (Admin)**.
2. Run `wsl --install` and restart the laptop when asked.
3. Download Docker Desktop from https://www.docker.com/products/docker-desktop and install it with default options (use WSL 2).
4. Start Docker Desktop and wait until it says **Engine running**.

Mac

Download Docker Desktop for your chip (Apple Silicon or Intel) from the same page, install, start it.

```
docker version
docker compose version
```

CHECKPOINT: PASS looks like this
Both commands print versions. `docker version` shows a **Server** section (engine running).

If it fails

What you see	Likely cause	Fix
WSL 2 installation is incomplete	WSL kernel missing	Run `wsl --update` in admin PowerShell, then restart Docker Desktop.
Virtualization not enabled	Disabled in BIOS	Restart, open BIOS (often F2/F10/Del), enable Intel VT-x or AMD SVM, save.
Laptop too slow / low RAM	Docker uses 2+ GB	Skip Part D. Part E works without Docker on your laptop.

A8 Create a GitHub account and an access token about 10 min

Why: The cloud server downloads your private code from GitHub.

1. Sign up at https://github.com (use the email you set in A3). Turn on two-factor authentication.
2. Students: apply for the GitHub Student Developer Pack at https://education.github.com/pack (optional, gives free tools).
3. You will create the repository in step B2 and the server token in step E6.

CHECKPOINT: PASS looks like this
You can log in to github.com.

A9 Final install check about 3 min

Why: One place to confirm everything before touching the project.

```
java -version
git --version
temporal --version
docker version
```

Also open IntelliJ once more. Write the results in your notes.

CHECKPOINT: PASS looks like this
Java 21, Git 2.x, Temporal CLI 1.x, Docker with a Server section (if you did A7). IntelliJ opens.

If it fails

What you see	Likely cause	Fix
Any command not found	That tool's step not finished	Go back to that step and re-check its checkpoint.

Gate A
All A checkpoints PASS (A5 and A7 optional). Only then start Part B.

Part B. Get the project and prove it builds

B1 Unzip the project to a safe folder about 5 min

Why: Paths with spaces or cloud-sync folders (OneDrive, iCloud) cause strange build errors.

1. Create the folder `C:\projects` (Windows) or `~/projects` (Mac: in Terminal run `mkdir -p ~/projects`).
2. Right-click `aerosaga-v1.0.zip` > **Extract All** (Windows) or double-click (Mac). Extract into the projects folder.
3. Make sure the result is `C:\projects\aerosaga\pom.xml` and NOT `C:\projects\aerosaga\aerosaga\pom.xml`. If it is nested, move the inner folder up one level.

Windows

```
cd C:\projects\aerosaga
dir
```

Mac

```
cd ~/projects/aerosaga
ls -la
```

CHECKPOINT: PASS looks like this
You see `pom.xml`, `README.md`, `Dockerfile`, `docker-compose.yml`, `.env.example`, folders `src` and `deploy`.

If it fails

What you see	Likely cause	Fix
`cd` says path not found	Extracted somewhere else	Search the computer for `pom.xml`, note its folder, use that path.
`.env.example` or `.github` not visible in Explorer	Hidden files	They are there; `dir /a` or `ls -la` shows them. Explorer: View > Show > Hidden items.

B2 Start version control and push to a private GitHub repository about 15 min

Why: Requirement: proper version control. GitHub also runs all tests automatically on every push (CI).

```
git init -b main
git add .
git status
```

Read the `git status` list. It must contain `.env.example` but must NOT contain a file called `.env`. Then:

```
git commit -m "Import AeroSaga v1.0 project"
```

1. On github.com click **+** (top right) > **New repository**.
2. Name: `aerosaga`. Select **Private**. Do NOT tick README, .gitignore or license. Click **Create repository**.
3. Copy the HTTPS URL shown, e.g. `https://github.com/<your-user>/aerosaga.git`.

```
git remote add origin https://github.com/<your-user>/aerosaga.git
git push -u origin main
```

Windows: a browser window opens to sign in to GitHub (Git Credential Manager). Approve it. Mac: if asked for a password, GitHub needs a token, not your password. Easiest: `brew install gh`, run `gh auth login`, choose GitHub.com, HTTPS, login with browser; then run the push again.

Open your repository page, click the **Actions** tab. A run called **CI** starts. Wait 3 to 8 minutes.

CHECKPOINT: PASS looks like this
Repository page shows the files and a **Private** label.
Actions tab: CI run has a green tick. This is an independent proof that all 59 tests pass on a clean Linux machine.

If it fails

What you see	Likely cause	Fix
`remote origin already exists`	Ran the command twice	`git remote set-url origin https://github.com/<your-user>/aerosaga.git`
Authentication failed on push	Password used instead of token, or browser login cancelled	Windows: run the push again and finish the browser login. Mac: use `gh auth login` as above.
CI run is red	A test failed on GitHub	Click the run > **test** job > **Build and run all tests**. Scroll to the first `ERROR` or `FAILED`. Use the B4 fix table; it is the same test suite.
Actions tab says Actions disabled	Account setting	Repository Settings > Actions > General > Allow all actions.

B3 Open the project in IntelliJ about 20 min

Why: IntelliJ downloads all libraries (Maven) and lets you run tests with one click.

1. IntelliJ > **Open** > select the `aerosaga` folder (the one with `pom.xml`) > **OK**.
2. If asked, click **Trust Project**.
3. Wait. The bottom bar shows **Resolving dependencies / Indexing**. First time: 5 to 15 minutes (about 300 MB download).
4. **File > Project Structure > Project**: set **SDK** to **21** (Temurin). If missing: Add SDK > JDK > pick the JDK 21 folder. Language level: **21**. Click OK.
5. Open the **Maven** tool window (right edge, letter m icon). Click the circular **Reload All Maven Projects** button once.

CHECKPOINT: PASS looks like this
No progress bar running at the bottom.
Open `src/main/java/com/infotact/aerosaga/workflow/MissionWorkflowImpl.java`: no red underlines on `import io.temporal...` lines.

If it fails

What you see	Likely cause	Fix
Red `Cannot resolve symbol` on imports	Maven import not finished or failed	Maven panel > Reload. Then View > Tool Windows > Build, read the first error.
`Could not transfer artifact` / `PKIX path` / timeout	College Wi-Fi or proxy blocks Maven Central	Switch to mobile hotspot or home Wi-Fi and Reload. Maven needs https://repo.maven.apache.org.
`Project JDK is not defined` banner	SDK not set	Do item 4 again.

What you see	Likely cause	Fix
`Unsupported class file major version`	Project uses an older JDK	Set SDK 21 in Project Structure AND in Settings > Build > Build Tools > Maven > Runner > JRE = 21.

B4 Run all 59 tests (the real proof) about 5 min

Why: Tests prove the rules, the saga workflow (on Temporal's in-memory test server) and the full HTTP API with security and database.

1. Maven tool window > **aerosaga > Lifecycle** > double-click **test**.
2. Or in a terminal inside the project folder: `mvn test` (needs A5).
3. Wait 1 to 4 minutes. Scroll to the end of the output.

Expected end of output

```
[INFO] Tests run: 59, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

What the 59 tests cover:

Test class	Tests	What it proves
GeoAndRouteTest	6	Distance maths (checked against 111.195 km per degree), routes, no-fly zone geometry
MissionPlanValidatorTest	11	Every plan rule; all errors reported at once
DroneFleetTest	9	Best drone chosen, idempotent booking, ownership, clear failure reasons
AirspaceRegistryTest	5	Slot limit, idempotent reserve/release, temporary zones
FlightSimulatorTest	6	Battery drain, faults fire before the drone moves, return home
SimulationStateCodecTest	5	Save/restore of the simulated world, corrupt data rejected
MissionWorkflowTest	8	Real Temporal test server: happy path, 3 faults, abort, invalid plan, no drone, deterministic replay
MissionApiTest	9	HTTP: login required, CSRF required, 202/404/409/422/400 codes, mission stored in database

CHECKPOINT: PASS looks like this
`Tests run: 59, Failures: 0, Errors: 0, Skipped: 0` and `BUILD SUCCESS`.

If it fails

What you see	Likely cause	Fix
`COMPILATION ERROR` ... `cannot find symbol` in a Temporal or Spring class	A library API differs from what the code expects (this part was NOT VERIFIED before delivery)	Note the FIRST error: file, line, symbol. In IntelliJ open that line, place cursor on the red word, press **Alt+Enter** (Mac: Option+Enter) to see suggested imports. If no fix appears, send the first error block to your mentor or AI assistant. Do not edit other files.
`Could not find artifact ...`	Wrong version or network	Check network (B3). Confirm `pom.xml` versions: Spring Boot 4.1.1, Temporal 1.38.0. Maven panel > Reload.

What you see	Likely cause	Fix
`Tests run: 59, Failures: 1` (or more)	A behaviour differs	Open `target/surefire-reports/` and the `.txt` file of the failing class. Read the first `expected ... but was ...` line. It names the scenario.
MissionApiTest: `did not finish in time`	Slow laptop	In `MissionApiTest.java` change `i < 150` to `i < 600` in `waitForFinalPhase`. Re-run.
`Address already in use` during tests	Another program on a test port	Close other running apps (another AeroSaga, other Java apps) and re-run.
`OutOfMemoryError`	Low RAM	Close other apps. Settings > Build > Build Tools > Maven > Runner > VM options: `-Xmx1g`.

B5 Commit the first green build about 2 min

Why: Small, honest commits show steady progress.

```
git add .
git commit -m "Verified: all 59 tests pass locally"
git push
```

If there is nothing to commit (no changes), that is fine. Take a screenshot of the test result for your report.

CHECKPOINT: PASS looks like this
`git push` ends without errors; GitHub shows your latest commit message.

If it fails

What you see	Likely cause	Fix
`nothing to commit, working tree clean`	No files changed	Normal. Skip the commit.

Gate B
B4 shows 59 tests passing on your laptop AND the GitHub CI run is green. Only then start Part C.

Part C. Run it on your laptop

You will use two terminals: one for Temporal, one for the app (or IntelliJ). Keep both open while testing.

C1 Start the Temporal server about 3 min

Why: The app needs a Temporal server to run missions.

Open a NEW terminal, go to the project folder, then start Temporal with a data file (so data survives restarts):

Windows (Mac: cd ~/projects/aerosaga)

```
cd C:\projects\aerosaga
temporal server start-dev --db-filename temporal.db
```

Leave this window open. Open http://localhost:8233 in your browser.

CHECKPOINT: PASS looks like this
The terminal prints the server address `localhost:7233` and the Web UI address `http://localhost:8233`. The browser shows the Temporal UI with namespace **default** and an empty workflow list.

If it fails

What you see	Likely cause	Fix
`address already in use` 7233 or 8233	Temporal already running elsewhere	Windows: `netstat -ano | findstr :7233` shows the PID in the last column; stop it with `taskkill /PID <pid> /F`. Mac: `lsof -i :7233` then `kill <pid>`.
Windows Firewall popup	Temporal opens local ports	Click **Allow** (private networks).
Window closed by mistake	Server stopped	Run the same command again in the same folder. Data in temporal.db is kept.

C2 Start the application about 3 min

Why: Runs the API, dashboard, database migrations and the Temporal worker.

1. IntelliJ: open `src/main/java/com/infotact/aerosaga/AeroSagaApplication.java`.
2. Click the green triangle next to `public class AeroSagaApplication` > **Run**.
3. Or terminal: `mvn spring-boot:run` in the project folder.

Watch the Run window. Then open http://localhost:8080/api/ping in the browser.

CHECKPOINT: PASS looks like this
Log lines: `Successfully applied 2 migrations`, `No saved simulator state; starting with a fresh fleet`, `Temporal worker polling task queue 'aerosaga-missions'`, `Started AeroSagaApplication`. The ping page shows JSON with `"status":"UP"`.

If it fails

What you see	Likely cause	Fix
`Port 8080 was already in use`	Another app on 8080	Windows: `netstat -ano | findstr :8080` shows the PID in the last column; stop it with `taskkill /PID <pid> /F`. Mac: `lsof -i :8080` then `kill <pid>`. Or add `server.port: 8081` in application.yml.

What you see	Likely cause	Fix
`UNAVAILABLE` / `Connection refused: localhost/127.0.0.1:7233`	Temporal not running	Do C1 first, then restart the app.
`Database may be already in use`	Two copies of the app running	Stop all runs (red square in IntelliJ). Start one.
`Validate failed: Migrations have failed validation` (Flyway)	A migration file was edited after it ran	Undo the edit. For a fresh local start: stop app, delete the `data` folder (deletes local history), start again.
Startup error mentioning `aerosaga.temporal` or a placeholder	application.yml damaged	Restore it: `git checkout -- src/main/resources/application.yml`.

C3 Log in to the dashboard about 2 min

Why: Confirms security, static pages and the API work together.

1. Open http://localhost:8080 (use a private/incognito window for a clean login).
2. The browser shows a sign-in box. User `operator`, password `local-dev-only` (laptop defaults only).

CHECKPOINT: PASS looks like this
Map of Electronic City, Bengaluru with a blue dashed circle (operating area) and a red circle (HAL Airport no-fly zone).
Fleet panel: DR-01, DR-02, DR-03 all AVAILABLE at 100%. Top right: `Slots: 0/3` and a green **Online** chip.

If it fails

What you see	Likely cause	Fix
Sign-in box keeps returning	Wrong user or password	Type exactly `operator` / `local-dev-only`. Browsers remember Basic login until fully closed; use a new private window.
Map is grey but panels work	No internet for map tiles (unpkg.com, openstreetmap.org)	Check internet. The app still works; only the background map is missing.
Red chip `Offline: ...`	App stopped or crashed	Check the IntelliJ Run window for errors (C2 table).

C4 Fly a normal mission about 5 min

Why: End-to-end proof of the happy path.

1. Click **Use demo route** (2 waypoints appear). Payload `1.0`. Fault: **None**.
2. Click **Launch mission**. The mission appears in the Missions list and is selected.
3. Watch phases: VALIDATING, RESERVING_AIRSPACE, ASSIGNING_DRONE, LOADING_PAYLOAD, IN_FLIGHT, DELIVERING, RETURNING, COMPLETED. At the default speed (20x faster than real) it takes about 1 minute.
4. In the Temporal UI (http://localhost:8233) click the workflow with the same id. Open **Event History**: you will see each activity and the flight timers.

CHECKPOINT: PASS looks like this
Final pill: **COMPLETED**. Drone **DR-03**, Legs 2/2, flown about 16.8 km.
Fleet: DR-03 AVAILABLE with about 58% battery. Slots 0/3. Temporal UI status: Completed.

If it fails

What you see	Likely cause	Fix
Stays in SUBMITTED	Worker not running or wrong task queue	App log must show `Temporal worker polling task queue`. Restart app after Temporal (C1, C2).
422 error: plan breaks rules	Waypoint outside area or in a no-fly zone	Read the red error list under Launch; move the waypoint.
403 when launching	Missing security token (page loaded before app restart)	Refresh the page (F5) and launch again.

C5 Test every fault scenario about 15 min

Why: Shows the saga undo logic and automatic retries. These are your strongest demo moments.

Use the demo route for each run. Recharge drones in the Fleet panel when their battery is low.

Fault (dropdown)	What you should see	Temporal UI evidence
Battery cell fault	IN_FLIGHT, then COMPENSATING with orange UNDO lines in this order: emergency return, unloading payload, releasing drone, releasing airspace. Final **ROLLED_BACK**, failure `LOW_BATTERY`.	Failed `FlyLeg` activity, then undo activities after it.
No-fly zone appears	Fails at first leg, final **ROLLED_BACK**, `GEOFENCE_BREACH`, 0 legs, 0 km. An orange zone `TFR-M-...` appears on the map and in the zones list.	FlyLeg failed once (non-retryable).
Flaky telemetry	Short pause on leg 1, then continues. Final **COMPLETED**.	Open the FlyLeg activity: attempt 3 succeeded after 2 failures.
None + Abort button	Launch, wait for IN_FLIGHT, click **Abort mission**, confirm. Final **ABORTED**, `OPERATOR_ABORT`, all UNDO lines.	A `WorkflowExecutionSignaled` event named `abort`.
Rule check	Click a waypoint inside the red airport circle and launch.	Nothing starts. Error list shows `inside no-fly zone 'HAL Airport'` (HTTP 422).

After the no-fly zone test, click **Lift** next to the TFR zone, otherwise later missions to that point will also fail.

CHECKPOINT: PASS looks like this
Each row ends exactly as described. After every run: all drones AVAILABLE (none stuck ASSIGNED/IN_FLIGHT) and Slots 0/3.

If it fails

What you see	Likely cause	Fix
`NO_DRONE_AVAILABLE` on a normal mission	Batteries drained by earlier tests	Click **Recharge** on each drone, launch again.

What you see	Likely cause	Fix
A drone stays ASSIGNED after a mission ended	An undo step failed	Check app log for `Compensation failed`. Final phase would be COMPENSATION_FAILED. Report with the mission id.
Abort button greyed out	Mission already delivering/returning/finished	Abort is only allowed before delivery. Launch again and abort earlier.

C6 Crash tests: prove durability about 10 min

Why: The main reason to use Temporal. Missions must survive restarts of the engine and of the app.

C6a. Restart the Temporal server during a flight

1. Launch a normal mission. Wait for IN_FLIGHT.
2. In the Temporal terminal press **Ctrl+C**. The server stops. The dashboard keeps showing the last stored phase.
3. Wait 15 seconds. Run `temporal server start-dev --db-filename temporal.db` again in the same folder.

Expected: within about 30 seconds the mission continues and ends **COMPLETED**.

C6b. Restart the application during a flight

1. Launch a normal mission. Wait for IN_FLIGHT.
2. In IntelliJ click the red **Stop** square. Wait 10 seconds. Click **Run** again.
3. Log shows `Restored simulator state: 3 drones, 1 slots in use, 0 temporary zones`.

Expected: the mission continues where it stopped and ends **COMPLETED**. If you stopped the app in the middle of an activity, Temporal waits for that activity's 30 second timeout and then retries it. A pause up to 30 seconds is normal.

> **CHECKPOINT: PASS looks like this**
> Both missions end COMPLETED. Temporal UI shows one continuous history per mission. All drones AVAILABLE afterwards.

If it fails

What you see	Likely cause	Fix
After C6a everything is gone in the Temporal UI	Server started without `--db-filename temporal.db` or in another folder	Always start it from the project folder with the same command.
After C6b mission ends ROLLED_BACK with `No drone assigned`	Simulator state not restored	Check the startup log for `Restored simulator state`. If it says `fresh fleet`, the `data` folder was deleted or a different database was used.
Mission waits a long time after restart	Activity timeout (30 s) plus retry backoff	Wait up to 1 minute before deciding it failed.

C7 Call the REST API directly about 10 min

Why: Your mentor may test the API without the dashboard. Also good for the report.

Create a file `mission.json` in the project folder with this content:

mission.json

```
{"operator":"api-user","payloadKg":1.0,"scenario":"NONE",
 "waypoints":[{"name":"Bommasandra","lat":12.8169,"lon":77.6950},
              {"name":"Hosur Road Hub","lat":12.8700,"lon":77.6400}]}
```

Step 1: get a security token. This also saves a cookie in `cookies.txt`:

Windows (Mac: use curl instead of curl.exe in all lines)

```
curl.exe -s -u operator:local-dev-only -c cookies.txt http://localhost:8080/api/csrf
```

Copy the long `token` value from the reply. Step 2: start a mission (replace TOKEN):

Windows PowerShell (the backtick ` at line end continues the command)

```
curl.exe -s -u operator:local-dev-only -b cookies.txt `
  -H "X-XSRF-TOKEN: TOKEN" -H "Content-Type: application/json" `
  --data "@mission.json" http://localhost:8080/api/missions
```

Mac (the backslash at line end continues the command)

```
curl -s -u operator:local-dev-only -b cookies.txt \
  -H "X-XSRF-TOKEN: TOKEN" -H "Content-Type: application/json" \
  --data "@mission.json" http://localhost:8080/api/missions
```

Step 3: read status, history, fleet (replace M-XXXXXXXX with the `missionId` from step 2):

```
curl.exe -s -u operator:local-dev-only http://localhost:8080/api/missions/M-XXXXXXXX
curl.exe -s -u operator:local-dev-only http://localhost:8080/api/missions/M-XXXXXXXX/events
curl.exe -s -u operator:local-dev-only http://localhost:8080/api/fleet
```

CHECKPOINT: PASS looks like this
Step 2 returns JSON with `"phase"` and a `missionId` starting with `M-`. Step 3 shows the mission progressing to COMPLETED.

If it fails

What you see	Likely cause	Fix
401 Unauthorized	Wrong user/password in `-u`	Use `operator:local-dev-only` (laptop) or your .env values (server).
403 Forbidden on POST	Token missing or not matching the cookie	Repeat step 1, copy the new token exactly, keep `-b cookies.txt`.
400 `Request body is not valid JSON`	File not found or broken JSON	Run the command in the folder that contains mission.json. Check quotes and commas.

C8 Code tour: understand what you will present about 2 to 3 hours

Why: You must be able to explain the design in the review. Read in this order.

Order	File / package	Key idea to explain
1	`domain/GeoPoint`, `RoutePlan`, `NoFlyZone`	Pure geography. Haversine distance. Route = legs + return leg.
2	`domain/MissionPlanValidator`	All business rules in one place. Collects every error. No I/O, easy to test.
3	`fleet/DroneFleet`, `BatteryModel`	Picks drone with most spare battery. Every method idempotent per mission id.

Order	File / package	Key idea to explain
4	`fleet/FlightSimulator`	Safety checks in order. Faults are checked before the drone moves.
5	`workflow/MissionWorkflow`	Contract: one workflow method, one signal (abort), one query (status).
6	`workflow/MissionWorkflowImpl`	The saga. Undo is registered BEFORE each step. Retry policies: steps 5 tries, undo 20 tries.
7	`workflow/MissionActivitiesImpl`	Business errors become non-retryable failures; telemetry loss stays retryable. State saved after each step.
8	`persistence/*`, `db/migration/*`	Mission history for reports; simulator state for crash recovery; Flyway owns the schema.
9	`service/MissionService`	Validates first, saves SUBMITTED row, starts workflow with mission id as workflow id.
10	`config/SecurityConfig`, `TemporalConfig`	Login + CSRF. Client, worker, clean start/stop.
11	`static/app.js`	Polls every 1.5 s, escapes all text (prevents script injection), sends the CSRF header.

The heart of the saga (from `MissionWorkflowImpl`). Notice that the undo is added before the step runs:

```
saga.addCompensation(() -> {
    note("UNDO: releasing drone");
    undo.releaseDrone(missionId);
});
DroneSnapshot drone = steps.assignDrone(missionId, request.payloadKg(), route.totalKm());
```

Why before? The step might succeed on the server but its reply might be lost (timeout). If the undo were added after, a lost reply would leave a drone booked forever. Undo steps are safe even if the step never happened.

Rules for workflow code (Temporal replays it)

- No `System.currentTimeMillis()`, `Thread.sleep`, random numbers, files, network or database inside the workflow class. Use `Workflow.currentTimeMillis()`, `Workflow.sleep()`, `Workflow.await()` and activities.
- The route is computed by an activity (`planRoute`) and its result is recorded, so replay never recomputes it.
- The test `historyReplaysDeterministically` proves the code replays the same way.

CHECKPOINT: PASS looks like this
You can explain, without notes: what a saga is, why undo is registered first, what a signal and a query are, and why missions survive restarts.

C9 Commit progress about 2 min

Why: Keep your history up to date.

```
git add .
git commit -m "Local run verified: all scenarios and crash tests"
git push
```

`cookies.txt` is ignored by Git (it holds a temporary token). Delete it after testing.

CHECKPOINT: PASS looks like this
GitHub shows the new commit.

Gate C
C4, C5 (all 5 rows), C6a and C6b PASS. Screenshots saved. Then start Part D or go straight to Part E.

Part D. Run the production stack on your laptop (Docker)

Optional but recommended: you test the exact containers that will run on the server, before paying for a server. Needs A7.

D1 Create the secrets file about 5 min

Why: The stack reads passwords and the domain from `.env`. This file is never committed.

Windows (in the project folder)

```
Copy-Item .env.example .env
notepad .env
```

Mac

```
cp .env.example .env
open -e .env
```

Set these values for the laptop test, then save:

```
DOMAIN=localhost
DB_USER=aerosaga
DB_PASSWORD=laptop-db-pass-123
AEROSAGA_USER=operator
AEROSAGA_PASSWORD=laptop-ui-pass-123
AEROSAGA_TIME_SCALE=0.05
```

Check

```
git status
```

CHECKPOINT: PASS looks like this
`.env` exists. `git status` does NOT list `.env` (it is ignored).

If it fails

What you see	Likely cause	Fix
`git status` lists `.env`	.gitignore damaged	Restore it: `git checkout -- .gitignore`. Never commit `.env`.
Notepad saved as `.env.txt`	Windows added an extension	In Notepad: Save As, file type **All files**, name `.env`.

D2 Build and start all containers about 15 min

Why: Builds the app image and starts Caddy, app, PostgreSQL and Temporal.

Stop the laptop app and the local Temporal server from Part C first (they use the same ports).

```
docker compose up -d --build
docker compose ps
```

First build: 5 to 15 minutes (downloads Maven libraries inside Docker). Then follow the app log:

Press Ctrl+C to stop following (containers keep running)

```
docker compose logs -f app
```

CHECKPOINT: PASS looks like this
`docker compose ps` lists `caddy`, `app`, `postgres`, `temporal`, all **running** (postgres also **healthy**).
App log ends with `Started AeroSagaApplication`.

If it fails

What you see	Likely cause	Fix
`set DB_USER in .env` (or other variable)	Missing value in .env	Fill every line in .env, save, run again.
`ports are not available` / bind 80 or 443	Another program uses web ports (IIS, Skype, other server)	Stop that program, or skip Part D.
Build fails with `Killed` or out of memory	Docker Desktop memory too low	Docker Desktop > Settings > Resources > Memory 4 GB or more. Run again.
App keeps restarting; log says connection refused 7233	Temporal still starting	Wait 30 seconds; Docker restarts the app automatically. If it continues, check `docker compose logs temporal`.
`password authentication failed for user`	DB password changed after the first start (PostgreSQL keeps the first one)	Laptop only: `docker compose down -v` deletes all stack data, then `up -d --build` again.

D3 Use the stack through HTTPS about 10 min

Why: Same path as production: browser to Caddy (HTTPS) to app.

1. Open https://localhost . The browser warns about the certificate (Caddy made a local one). Click **Advanced** > **Proceed**.
2. Log in with AEROSAGA_USER / AEROSAGA_PASSWORD from your .env.
3. Fly one normal mission and one Battery cell fault mission.
4. Temporal UI: http://localhost:8233

CHECKPOINT: PASS looks like this
Dashboard works over https://localhost. Both missions end as expected (COMPLETED, ROLLED_BACK).

If it fails

What you see	Likely cause	Fix
Login rejected	Using laptop defaults instead of .env values	Use the values from .env.
Page does not load	Caddy not running	`docker compose logs caddy` and check D2.

D4 Stop the stack about 1 min

Why: Free laptop memory when finished.

Stops containers. Keeps all data.

```
docker compose down
```

Irreversible command
`docker compose down -v` also deletes the database and Temporal data volumes. Use only to start completely fresh.

CHECKPOINT: PASS looks like this
`docker compose ps` shows nothing running.

Gate D
D2 and D3 PASS. You now know the production containers work.

Part E. Go live on a cloud server (Azure for Students)

Plan: one Ubuntu virtual machine (VM) in Azure Central India, Docker Compose, a free Azure DNS name, and an automatic HTTPS certificate from Caddy. Azure for Students gives US$100 credit for 12 months without a credit card (checked 14 Oct 2026, https://azure.microsoft.com/free/students).

Other providers work the same way
Any Ubuntu 24.04 server with 2 vCPU and 4 GB RAM, a public IP, and ports 22, 80, 443 open works. Steps E4 to E13 are identical; only E1 to E3 (creating the server and its DNS name) differ.
Azure screens change over time. If a button name differs, look for the same idea nearby.

E1 Activate Azure for Students about 15 min

Why: Free credit, no card.

1. Open https://azure.microsoft.com/free/students and click **Start free**.
2. Sign in or create a Microsoft account. Verify with your **college email**.
3. Open https://portal.azure.com . In the search bar type **Subscriptions**: you should see **Azure for Students**.
4. Search **Cost Management** > **Budgets** > **Add**: monthly budget US$40, alert at 80% to your email. This protects your credit.

CHECKPOINT: PASS looks like this
Subscription **Azure for Students** is listed and active. A budget alert exists.

If it fails

What you see	Likely cause	Fix
College email not accepted	Institution not recognised	Ask your college for the official email domain, or use another provider with student credit.

E2 Create the virtual machine about 15 min

Why: This is the server that runs everything.

Portal search bar: **Virtual machines** > **Create** > **Azure virtual machine**. Fill in:

Field	Value
Subscription / Resource group	Azure for Students / **Create new**: `aerosaga-rg`
Virtual machine name	`aerosaga-vm`
Region	**(Asia Pacific) Central India** (or South India)
Image	**Ubuntu Server 24.04 LTS - x64 Gen2**
Size	**Standard_B2s** (2 vCPU, 4 GiB). If not available, any size with 2 vCPU and 4 GiB.
Authentication type	**SSH public key**
Username	`azureuser`
SSH key source	**Generate new key pair**, key name `aerosaga-key`
Public inbound ports	Allow selected ports: **SSH (22), HTTP (80), HTTPS (443)**

Field	Value
Disks / Networking	Defaults

1. Click **Review + create**, then **Create**.
2. A popup asks to **Download private key and create resource**. Click it. Save `aerosaga-key.pem` in a safe folder, for example `C:\Users\<you>\.ssh\`. Without this file you cannot log in.
3. Wait for **Your deployment is complete** > **Go to resource**.

CHECKPOINT: PASS looks like this
VM overview shows Status **Running** and a **Public IP address**. The .pem file is saved on your laptop.

If it fails

What you see	Likely cause	Fix
Size not available / quota error	Student subscriptions limit some sizes or regions	Pick another size with 4 GiB (e.g. B2als_v2) or region South India.
Key download popup missed	Browser blocked it	Delete the VM (resource group) and create again; keys cannot be downloaded later.

E3 Give the server a DNS name about 5 min

Why: HTTPS certificates need a name, not just an IP number.

1. VM overview > click the **Public IP address** link.
2. Left menu **Settings > Configuration**.
3. **DNS name label**: `aerosaga-<yourname>` (lowercase, letters, numbers, hyphens). **Save**.
4. Copy the full name, e.g. `aerosaga-ganesh.centralindia.cloudapp.azure.com`. This is your DOMAIN.

Check from your laptop

```
nslookup aerosaga-ganesh.centralindia.cloudapp.azure.com
```

CHECKPOINT: PASS looks like this
`nslookup` returns your VM's public IP address.

If it fails

What you see	Likely cause	Fix
Label already taken	Names are global	Choose another label.
nslookup fails	DNS not updated yet	Wait 2 to 5 minutes and retry.

E4 Connect to the server with SSH about 5 min

Why: You control the server by typing commands in a remote terminal.

Windows PowerShell (the icacls lines fix key permissions once)

```
cd $HOME\.ssh
icacls aerosaga-key.pem /inheritance:r
icacls aerosaga-key.pem /grant:r "$($env:USERNAME):(R)"
ssh -i aerosaga-key.pem azureuser@aerosaga-ganesh.centralindia.cloudapp.azure.com
```

Mac

```
chmod 400 ~/.ssh/aerosaga-key.pem
ssh -i ~/.ssh/aerosaga-key.pem azureuser@aerosaga-ganesh.centralindia.cloudapp.azure.com
```

First time it asks `Are you sure you want to continue connecting?` Type `yes`.

CHECKPOINT: PASS looks like this
The prompt changes to `azureuser@aerosaga-vm:~$`. You are now typing on the server.

If it fails

What you see	Likely cause	Fix
`UNPROTECTED PRIVATE KEY FILE` / bad permissions	Key file readable by others	Run the icacls (Windows) or chmod (Mac) lines again.
`Permission denied (publickey)`	Wrong user or wrong key	User must be `azureuser`; use the .pem downloaded for this VM.
`Connection timed out`	Port 22 closed or VM stopped	VM must be Running. VM > Networking > inbound rule for port 22 must exist.

E5 Prepare the server (updates and Docker) about 10 min

Why: Install security updates and the container engine.

Run on the server, one line at a time:

```
sudo apt update && sudo apt -y upgrade
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
exit
```

Connect again (E4 ssh command), then check Docker and add a 2 GB swap file (extra safety for the build):

```
docker run --rm hello-world
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
free -h
```

CHECKPOINT: PASS looks like this
`hello-world` prints `Hello from Docker!`. `free -h` shows Swap 2.0Gi.

If it fails

What you see	Likely cause	Fix
`permission denied ... docker.sock`	Group change not active yet	Type `exit`, connect again, retry.
apt asks about a config file during upgrade	Ubuntu package question	Press Enter to keep the default.
`fallocate: swapfile: Text file busy`	Swap already exists	Fine, skip the swap lines.

E6 Download your code to the server about 10 min

Why: The server clones your private GitHub repository using a read-only token.

1. github.com > your photo > **Settings** > **Developer settings** > **Personal access tokens** > **Fine-grained tokens** > **Generate new token**.
2. Name `aerosaga-server`, expiry 90 days, **Only select repositories**: `aerosaga`. Permissions: **Contents: Read-only**. Generate and copy the token (shown once).

On the server

```
git clone https://github.com/<your-user>/aerosaga.git
cd aerosaga
ls
```

When asked: Username = your GitHub username, Password = paste the token (nothing shows while pasting; press Enter).

CHECKPOINT: PASS looks like this
`ls` shows `docker-compose.yml`, `Dockerfile`, `src`, `deploy`.

If it fails

What you see	Likely cause	Fix
`Authentication failed`	Token wrong, expired or no repo access	Create a new token with access to `aerosaga` and Contents read.
`Repository not found`	Typo in URL or token lacks access	Copy the URL from the GitHub page; check token repository selection.

E7 Create production secrets about 5 min

Why: Strong, unique passwords. The app refuses to start if any value is missing.

Run twice: copy the two random strings

```
openssl rand -base64 24
openssl rand -base64 24
```

```
cp .env.example .env
chmod 600 .env
nano .env
```

In nano: move with arrow keys, edit the values, then **Ctrl+O**, **Enter** to save, **Ctrl+X** to exit.

Example .env

```
DOMAIN=aerosaga-ganesh.centralindia.cloudapp.azure.com
DB_USER=aerosaga
DB_PASSWORD=<first random string>
AEROSAGA_USER=operator
AEROSAGA_PASSWORD=<second random string>
AEROSAGA_TIME_SCALE=0.05
```

Security
Store the two passwords in a password manager. Never paste them in chat, email or GitHub.
Decide DB_PASSWORD now. PostgreSQL saves it on first start; changing it later needs F4.

CHECKPOINT: PASS looks like this
`cat .env` shows your real values (no REPLACE_ text). `ls -l .env` starts with `-rw-------`.

If it fails

What you see	Likely cause	Fix
nano not found	Minimal image	`sudo apt install -y nano`

E8 Build and start the production stack about 15 min

Why: Same as Part D, now on the server.

```
docker compose up -d --build
docker compose ps
docker compose logs --tail 50 app
docker compose logs --tail 30 caddy
```

CHECKPOINT: PASS looks like this
All four services running; postgres healthy.
App log: `Started AeroSagaApplication`. Caddy log contains `certificate obtained successfully` for your DOMAIN.

If it fails

What you see	Likely cause	Fix
Caddy: `challenge failed` / no certificate	Port 80 or 443 blocked, or DOMAIN typo	VM > Networking: inbound rules for 80 and 443. Check DOMAIN in .env equals E3 name. Then `docker compose restart caddy`.
App: `Could not resolve placeholder 'DB_URL'` or similar	Missing value	Fill .env, then `docker compose up -d`.
Build stops with `Killed`	Out of memory	Make sure swap from E5 is active (`free -h`); run the command again.
App restarts in a loop	Read the reason	`docker compose logs --tail 100 app`, find the first `ERROR` or `Caused by`. Use Appendix A.

E9 Verify the live site about 10 min

Why: Production smoke test: real URL, real certificate, real login.

1. On your laptop open `https://<DOMAIN>`. The padlock icon must show (valid certificate, no warning).
2. Log in with AEROSAGA_USER / AEROSAGA_PASSWORD.
3. Fly one normal mission and one Battery cell fault mission.
4. Open `https://<DOMAIN>/actuator/health` (public): it shows `{"status":"UP"...}`.

Must FAIL (timeout or refused): Temporal UI is not public

```
curl.exe -m 5 http://<DOMAIN>:8233
```

CHECKPOINT: PASS looks like this
Padlock valid, login works, both missions end correctly, health UP, port 8233 NOT reachable from the internet.

If it fails

What you see	Likely cause	Fix
Browser warns about certificate	Caddy did not get a certificate	See E8 Caddy row.

What you see	Likely cause	Fix
Port 8233 reachable from laptop	Compose file changed	In docker-compose.yml the temporal port must be `127.0.0.1:8233:8233`. Fix, `docker compose up -d`.

E10 Open the Temporal UI safely (SSH tunnel) about 3 min

Why: Admins see workflow histories without exposing the UI to the internet.

On your laptop (keep this window open)

```
ssh -i aerosaga-key.pem -L 8233:localhost:8233 azureuser@<DOMAIN>
```

Now open http://localhost:8233 on your laptop. Close the window to close the tunnel.

CHECKPOINT: PASS looks like this
The Temporal UI shows the missions you flew in E9.

If it fails

What you see	Likely cause	Fix
`bind: address already in use`	Local Temporal from Part C still running	Stop it (Ctrl+C in its terminal) or use `-L 9233:localhost:8233` and open port 9233.

E11 Automatic daily backups and a restore test about 15 min

Why: A backup you never restored is not a backup.

On the server, in ~/aerosaga

```
bash deploy/backup.sh
ls -lh backups/
```

Schedule it every day at 02:30 UTC (08:00 India time):

Choose nano if asked; add this line at the end, save

```
crontab -e
```

```
30 2 * * * cd /home/azureuser/aerosaga && bash deploy/backup.sh >> backups/backup.log 2>&1
```

Restore test (do it once now, while data is not important). It replaces the database with the backup:

Replace <file> with the name from ls; replace aerosaga with your DB_USER if different

```
docker compose stop app
docker compose exec -T postgres psql -U aerosaga -d postgres -c "DROP DATABASE aerosaga;"
docker compose exec -T postgres psql -U aerosaga -d postgres -c "CREATE DATABASE aerosaga;"
gunzip -c backups/<file>.sql.gz | docker compose exec -T postgres psql -U aerosaga -d aerosaga
docker compose start app
```

Irreversible
DROP DATABASE deletes current mission history. Only restore when you mean it.

Copy backups off the server sometimes (from your laptop):

```
scp -i aerosaga-key.pem azureuser@<DOMAIN>:aerosaga/backups/*.sql.gz .
```

CHECKPOINT: PASS looks like this
`backups/` contains a `.sql.gz` file larger than a few KB. After the restore test, the dashboard still lists your missions. `crontab -l` shows the line.

If it fails

What you see	Likely cause	Fix
`Backup looks empty`	Database name or user wrong	Check DB_USER in .env; `docker compose ps` postgres must be running.
Restore: `database is being accessed by other users`	App still connected	Run `docker compose stop app` first.

E12 Uptime monitoring and cost control about 10 min

Why: Know about problems before your mentor does; never run out of credit by surprise.

1. Create a free account at https://uptimerobot.com . Add monitor: type **HTTP(s) - Keyword**, URL `https://<DOMAIN>/actuator/health`, keyword `UP`, interval 5 minutes, alert to your email.
2. Azure portal > Cost Management: check spend weekly. The budget alert from E1 emails you at 80%.
3. When the project is over: VM > **Stop** (stops compute billing; small storage/IP charges continue). To remove everything: delete resource group `aerosaga-rg` (irreversible, deletes server and data; take a backup copy first).

CHECKPOINT: PASS looks like this
UptimeRobot shows the monitor **Up**. Azure shows your current cost.

E13 Release gate (go-live checklist) about 20 min

Why: One table that proves the product is live and safe. Put it in your report.

#	Check	How	Result
1	All tests pass	GitHub Actions latest run green	PASS / FAIL
2	HTTPS valid	Padlock on https://DOMAIN	
3	Login required	Private window: dashboard asks for login	
4	Wrong password rejected	Try a wrong password	
5	Normal mission	COMPLETED	
6	Fault mission rolls back	Battery fault: ROLLED_BACK, drones AVAILABLE	
7	Abort works	ABORTED, drones AVAILABLE	
8	Survives app restart	`docker compose restart app` during a flight; mission completes	
9	Temporal UI private	Port 8233 unreachable from laptop	
10	Secrets safe	`.env` not on GitHub; chmod 600	
11	Backup + restore tested	E11	
12	Monitoring	UptimeRobot Up	

CHECKPOINT: PASS looks like this
All 12 rows PASS. Any FAIL in rows 2, 3, 4, 9, 10 blocks go-live until fixed.

Gate E
E13 complete with all rows PASS. Your product is live. Send the URL and a test login to your mentor privately.

Part F. Operate the live product (runbook)

Commands run on the server inside `~/aerosaga` unless stated otherwise.

F1. Deploy an update

1. On the laptop: change code, run all tests (B4), commit, push. Wait for green CI.
2. On the server: `git pull` then `docker compose up -d --build`.
3. Smoke test: rows 2, 5, 6 of E13.

> **Rule for workflow code changes**
> Temporal replays running missions with the NEW code. If you change the order or number of steps in `MissionWorkflowImpl` while missions are running, those missions fail with a non-determinism error.
> Safe way: wait until no mission is running (all final phases), then deploy. Professional way for later: `Workflow.getVersion(...)` to keep old and new paths side by side.
> Changes to activities, API, dashboard and database are safe at any time (database changes only through a new Flyway file V3__...).

F2. Roll back a bad update

```
git log --oneline -5          # find the last good commit id
git checkout <good-commit-id>
docker compose up -d --build
# later, to return to the newest code:
git checkout main
```

F3. Logs, health, resources

```
docker compose ps                       # are all services up?
docker compose logs --tail 200 app      # last app messages
docker compose logs -f app              # follow live (Ctrl+C to stop)
docker compose restart app              # restart only the app
df -h        # disk space (keep below 80%)
free -h      # memory
docker system prune -f                  # remove old unused images (safe)
```

F4. Change passwords

- Dashboard password: edit `AEROSAGA_PASSWORD` in `.env`, then `docker compose up -d app`.
- Database password: PostgreSQL keeps the old one inside its data. First change it inside the database, then in `.env`:

```
docker compose exec postgres psql -U aerosaga -d aerosaga
# inside psql type the next line, then \q to quit:
ALTER USER aerosaga WITH PASSWORD 'NEW_PASSWORD';
nano .env      # set DB_PASSWORD=NEW_PASSWORD
docker compose up -d app
```

F5. Incident checklist (site down)

1. Is the VM running? (Azure portal). If stopped, Start it.
2. SSH in. `docker compose ps`: which service is not running?
3. `docker compose logs --tail 100 <service>`: read the first ERROR.
4. Disk full? `df -h`. If above 95%: `docker system prune -f`, delete old backups.
5. Restart the broken service: `docker compose up -d`.

6. Verify with E13 rows 2 and 5. Write down cause, fix and time for your report.

F6. Weekly routine (10 minutes)

- `sudo apt update && sudo apt -y upgrade` (security updates), then `sudo reboot` if it says a restart is required. Docker restarts all services automatically.
- Check `backups/backup.log` and that new backup files appear daily.
- Check Azure cost and UptimeRobot history.

Part G. Internship deliverables

The assignment letter asks for: Source Code, Project Documentation, Project Report, Presentation (PPT), Model Evaluation Results (if applicable) and Final Project Demonstration. Model evaluation is NOT APPLICABLE: AeroSaga has no machine learning model. Say so in the report in one line.

Deliverable	What to submit	Source in this guide
Source code	Private GitHub repo link (give mentor access) or zip of the repo	B2, F1
Documentation	README.md + this guide + API table (Appendix E) + architecture (1.4)	Repo + G1
Project report	PDF/Word, 15 to 25 pages, structure in G2	G2
Presentation	12 slides, structure in G3	G3
Model evaluation	Not applicable (no ML model)	-
Demonstration	Live site or laptop, 5 to 7 minute script in G4	G4

G1 Documentation package about 1 hour

Why: Evaluators check documentation standards.

- README.md (already in repo): purpose, quick start, project map, API, production, limits.
- Add a `docs/` folder with: this guide, architecture diagram screenshot, and your release gate table (E13).
- Every Java class has a short comment saying what it does and why. Keep that style when you change code.

CHECKPOINT: PASS looks like this
Repo has README.md and docs/ with the guide and diagram.

G2 Write the project report about 1 to 2 days

Why: The report is graded on clarity, evidence and honesty.

Chapter	What to write	Evidence to include
1. Abstract	5 to 7 lines: problem, approach (Temporal + saga), result (live product, 59 tests)	-
2. Introduction	Drone delivery, why long-running missions need reliability	-
3. Problem statement and objectives	Missions must never leave drones or airspace half-booked; must survive crashes	Requirement list (Appendix C)
4. Technology	Java 21, Spring Boot, Temporal (workflow, activity, signal, query), saga pattern, PostgreSQL, Docker, Caddy	Glossary 1.3
5. System design	Architecture, mission flow, data model (3 tables), API design, security design	Diagrams 1.4 and 1.1
6. Implementation	Package by package (C8 table). Explain undo-before-step and idempotency	Code excerpts (short)

Chapter	What to write	Evidence to include
7. Testing	Unit, workflow, API tests; fault scenarios; crash tests; CI	Test count screenshot, C5 table with your results, Temporal UI screenshots
8. Deployment	Docker Compose, HTTPS, secrets, backups, monitoring	E13 table filled in
9. Results	What works, timings, screenshots of dashboard	Screenshots (G5)
10. Limitations and future work	Single-node Temporal; simulated drones; single app instance; ideas: Temporal Cloud, real telemetry, workflow versioning, metrics dashboards	-
11. Conclusion	What you learned	-
References	Temporal docs, Spring Boot docs, saga pattern articles	Links

Write in your own words. Do not paste this guide into the report.

CHECKPOINT: PASS looks like this
Report draft sent to mentor by 6 Nov.

G3 Build the presentation (12 slides) about half a day

Why: Final review and presentation are part of the evaluation.

Slide	Title	Content
1	AeroSaga	Name, role, mentor, dates
2	Problem	Long missions + failures = stuck drones, double bookings
3	Goal	Reliable mission control that always ends clean
4	Architecture	Diagram 1.4
5	What is Temporal	Workflow, activity, worker; durable timers; replay
6	Saga pattern	Diagram 1.1: steps and undo steps
7	Mission flow	Phases and the dashboard screenshot
8	Fault scenarios	C5 table with results
9	Testing	59 tests, CI, crash tests
10	Deployment	Docker, HTTPS, backups, monitoring
11	Demo	Live link
12	Learnings and future work	3 learnings, 3 next steps

CHECKPOINT: PASS looks like this
12 slides, one idea each, more pictures than text.

G4 Demo script (6 minutes) about 2 hours

Why: A rehearsed demo shows confidence. Practise twice.

Time	Do	Say
0:00	Open dashboard, show map, zones, fleet	Operating area, airport no-fly zone, 3 drones
0:45	Use demo route, Launch (None)	Validation, booking, drone choice by spare battery
1:45	Open Temporal UI history	Every step recorded; timers are durable
2:30	Launch Battery cell fault	Watch UNDO lines run newest first
3:30	Launch normal, Abort during flight	Signal into a running workflow
4:15	Launch normal, `docker compose restart app` (or Stop/Run in IntelliJ)	Mission continues after restart
5:15	Show test results and CI	59 automated tests on every push
5:45	Questions	-

Backup plan: record a screen video of the whole demo the day before, in case the network fails.

CHECKPOINT: PASS looks like this
Demo runs within 7 minutes twice in a row without help.

G5 Screenshot checklist about 30 min

Why: Evidence for report and slides.

- Test result `Tests run: 59, Failures: 0` (IntelliJ) and green GitHub Actions run.
- Dashboard: COMPLETED mission with timeline; ROLLED_BACK mission with UNDO lines; ABORTED mission.
- Temporal UI: event history of a rolled-back mission; FlyLeg activity with attempt 3 (flaky).
- Live site with padlock; `docker compose ps` on server; backup file list.

CHECKPOINT: PASS looks like this
All screenshots saved in `docs/screenshots/` with clear names.

G6 Final verification before submission about 1 hour

Why: Same audit a professional team does before calling work complete.

#	Question	Your answer
1	Do all 59 tests pass locally and in CI?	
2	Did every C5 scenario and both C6 crash tests end as expected?	
3	Is every E13 row PASS?	
4	Is every requirement in Appendix C mapped to code and evidence?	
5	Are secrets out of GitHub and passwords strong?	
6	Was a backup restored successfully at least once?	
7	Can a beginner follow README to run it?	
8	Are known limits stated honestly in the report?	
9	Is every commit yours and the history meaningful?	

Final label for your report: **VERIFIED COMPLETE** only if all rows are yes with evidence. Otherwise **PARTIALLY COMPLETE** and list what remains.

CHECKPOINT: PASS looks like this
All 9 answers are yes, with evidence.

G7 Likely review questions (prepare answers) about 1 hour

Why: Interviewers ask these about workflow projects.

Question	Short answer
Why Temporal and not a simple database status column?	Temporal records every step, retries failures, keeps durable timers and resumes after crashes. A status column needs all of that hand-written.
What is a saga?	A long transaction split into steps, each with an undo step. On failure, undo steps run in reverse order.
Why register the undo before the step?	The step can succeed but its reply can be lost. Registering first guarantees cleanup. Undo steps are idempotent.
Why must workflow code be deterministic?	Temporal rebuilds state by replaying history through the code. Different decisions on replay would corrupt the mission.
Which failures are retried?	Technical ones (telemetry loss, timeouts). Business ones (low battery, no drone, geofence) are marked non-retryable.
Signal vs query?	Signal changes a running workflow (abort). Query only reads its state (status).
How do missions survive an app restart?	Temporal keeps workflow progress; our simulator state is saved to PostgreSQL after each step and restored at startup.
How is the app secured?	HTTPS via Caddy, login required, CSRF token on every change, secrets in .env, Temporal UI not public, non-root container.
What would you improve?	Temporal Cloud for high availability, workflow versioning, real drone telemetry, metrics and alerts dashboards.

CHECKPOINT: PASS looks like this
You can answer each question in your own words in under 30 seconds.

Appendix A. Troubleshooting index

First rule: find the FIRST error, not the last. Later errors are usually caused by the first one. Change one thing at a time, then re-run the same check.

Symptom	Go to
Command not recognised (java, git, temporal, mvn, docker)	A2 to A7 fix tables; open a new terminal
Libraries do not download / PKIX / timeout	B3
Compile error or failing test	B4
Port already in use (8080, 7233, 8233, 80, 443)	C1, C2, D2
App cannot connect to Temporal (UNAVAILABLE, 7233)	C1 then C2
Mission stuck in SUBMITTED	C4
NO_DRONE_AVAILABLE	C5: recharge drones
Missions to one point always fail with GEOFENCE_BREACH	C5: lift the TFR zone
403 when launching or aborting	C4: refresh page (new security token)
Mission lost after restart	C6
Docker build killed / out of memory	D2, E5 swap, E8
No HTTPS certificate on server	E8, E9
password authentication failed (PostgreSQL)	D2 (laptop), F4 (server)
Non-determinism error after deploying	F1 rule for workflow code
Site down	F5

How to ask for help (copy this)

```
Step: <step id, e.g. B4>
What I ran: <exact command or button>
Expected: <checkpoint text>
What I got (FIRST error, full lines):
<paste>
What I already tried:
<list>
OS: <Windows 11 / macOS 15>   Java: <java -version first line>
```

Appendix B. Command cheat sheet

Task	Command
Run tests	`mvn test` (or Maven panel > Lifecycle > test)
Run app (laptop)	`mvn spring-boot:run` (or IntelliJ Run)
Start Temporal (laptop)	`temporal server start-dev --db-filename temporal.db`
Reset local data (laptop)	Stop app; delete `data` folder and `temporal.db*` files
Start stack	`docker compose up -d --build`
Status	`docker compose ps`
App logs	`docker compose logs --tail 200 app`
Restart app	`docker compose restart app`
Stop stack (keep data)	`docker compose down`
Backup now	`bash deploy/backup.sh`
Update server	`git pull && docker compose up -d --build`
Temporal UI from laptop	`ssh -i aerosaga-key.pem -L 8233:localhost:8233 azureuser@<DOMAIN>`
Commit and push	`git add . && git commit -m "message" && git push`

Appendix C. Requirements traceability

Source: Project Assignment Letter ITS/JD/1144 (stated) and the project title (inferred, marked *).

ID	Requirement	Where implemented	Evidence
REQ-001	Java application using OOP	All packages; interfaces `MissionJournal`, `MissionActivities`; encapsulated `DroneFleet`	B4
REQ-002	Database integration	`persistence/`, Flyway `db/migration/`	MissionApiTest: mission stored; E11
REQ-003	APIs and third-party libraries	REST `api/`; Temporal SDK; Spring Boot	MissionApiTest; C7
REQ-004	Testing, debugging, optimisation	59 tests; CI workflow	B4, B2 CI
REQ-005	Version control and code documentation	Git + GitHub; class comments; README	B2, G1
REQ-006	Progress updates to mentor	Weekly template	Section 2.1
REQ-007	Finish within 1 month	4-week plan	Section 2
REQ-008	Deliverables: code, docs, report, PPT, demo	Part G	G1 to G6
REQ-009	Original work	Own commits, own report text	0.3, G2
REQ-010 *	Temporal.io orchestration	`workflow/`, `config/TemporalConfig`	MissionWorkflowTest
REQ-011 *	Saga compensation	`MissionWorkflowImpl`	lowBatteryRollsBackInReverseOrder; C5
REQ-012 *	Autonomous drone simulation	`fleet/FlightSimulator`, `DroneFleet`	FlightSimulatorTest
REQ-013 *	Live control: abort and status	Signal `abort`, query `status`	operatorAbortMidFlightBringsDroneHome; C5
REQ-014 *	Survive crashes	Temporal + `SimulationStateStore`	C6a, C6b, E13 row 8; replay test
REQ-015 *	Secure access	`SecurityConfig`, Caddy HTTPS	apiRequiresLogin, postWithoutCsrfTokenIsForbidden; E9
REQ-016 *	Live deployment and operations	Dockerfile, compose, Caddy, backups	E8 to E13

Appendix D. Configuration reference

Variable / property	Default (laptop)	Production	Meaning
AEROSAGA_USER	operator	required in .env	Dashboard/API login name
AEROSAGA_PASSWORD	local-dev-only	required in .env	Dashboard/API password
DB_URL / DB_USER / DB_PASSWORD	H2 file in ./data	PostgreSQL (set by compose + .env)	Database connection
TEMPORAL_TARGET	localhost:7233	temporal:7233	Temporal server address
TEMPORAL_NAMESPACE	default	default	Temporal namespace
AEROSAGA_TIME_SCALE	0.05	0.05	1.0 real flight time; 0.05 is 20x faster
DOMAIN	-	required in .env	Public name for HTTPS
aerosaga.airspace.capacity	3	3	Max missions flying at once (application.yml)
Operating limits	Electronic City, 25 km, 10 waypoints, 40 km route, 5 kg, 20% reserve, HAL zone	same	`OperatingLimits.bengaluruDemo()`

Appendix E. REST API reference

All endpoints need login except ping and health. POST and DELETE also need the `X-XSRF-TOKEN` header from `/api/csrf` (see C7).

Method	Path	Success	Errors
POST	/api/missions	202 + live view	400 bad body, 422 rule broken, 503 Temporal down
GET	/api/missions	200 last 50	-
GET	/api/missions/{id}	200 live view	404
GET	/api/missions/{id}/events	200 history	404
POST	/api/missions/{id}/abort	202	404, 409 already ended
GET	/api/fleet	200 drones, slots, zones	-
POST	/api/fleet/{droneId}/recharge	204	404, 409 drone flying
DELETE	/api/airspace/zones/{name}	204	-
GET	/api/config	200 limits	-
GET	/api/csrf	200 token	-
GET	/api/ping, /actuator/health	200 (public)	-

Mission request body

Text after <- is an explanation only; do not type it

```
{
  "operator": "operator-1",     <- required, max 100 chars
  "payloadKg": 1.0,              <- required, 0 to 5 (fleet max)
  "scenario": "NONE",            <- NONE, LOW_BATTERY, GEOFENCE_BREACH, FLAKY_TELEMETRY
  "waypoints": [                 <- 1 to 10 stops
    {"name": "Bommasandra", "lat": 12.8169, "lon": 77.6950}
  ]
}
```

Sources checked for versions and setup (14 Oct 2026)

- Temporal Java SDK releases (1.38.0): https://github.com/temporalio/sdk-java/releases
- Temporal CLI setup: https://docs.temporal.io/cli/setup-cli ; Docker image tags: https://hub.docker.com/r/temporalio/temporal/tags
- Spring Boot (4.1.1) and starters: https://spring.io/projects/spring-boot ; https://docs.spring.io/spring-boot/reference/using/build-systems.html
- Spring Boot 4.0 migration guide: https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Migration-Guide
- Spring Security CSRF: https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html
- Azure for Students: https://azure.microsoft.com/free/students